

FinWise AI

AI-Powered Personal Financial Analysis and Smart Budget Assistant

A LangChain + Streamlit FinTech Programming Assignment
Course Module: Building LLM Applications with LangChain

IMPORTANT — EDUCATIONAL USE ONLY

The application built in this assignment is an **educational prototype**. It must **not** provide guaranteed investment advice, execute financial transactions, connect to real bank accounts, or claim that any financial outcome is guaranteed. Every screen must state that it is for informational purposes only and direct users to a qualified financial professional.

Total marks: 100 • Suggested time: 1–2 weeks • Submission: GitHub repository + demo

1. Assignment Title

"AI-Powered Personal Financial Analysis and Smart Budget Assistant" — internally code-named **FinWise AI**.

You will build a Streamlit web app where users enter monthly income and expenses, Python computes key financial ratios, and a LangChain-powered LLM produces structured, educational budgeting insights.

2. Industry Scenario

A FinTech startup wants to build an AI-powered prototype called **FinWise AI**. Many individuals struggle to understand where their money goes and how to improve monthly planning. The company wants a web app where users enter income and expenses and receive personalized financial insights.

3. Business Problem

People often lack a clear, structured view of their spending. Generic advice online rarely reflects an individual's actual numbers. FinWise AI should:

- Analyse income and expenses and categorize spending.
- Identify high-expense areas and calculate savings potential.
- Generate a personalized budget strategy and improvement suggestions.
- Identify potential financial-risk patterns.
- Produce a clear monthly financial summary — all for education only.

4. Learning Objectives

By completing this assignment you will be able to:

- Integrate an OpenAI chat model with **ChatOpenAI**.
- Combine **deterministic Python calculations** with LLM-generated insight.
- Design reusable prompts with **PromptTemplate** and **ChatPromptTemplate**.
- Work with **SystemMessage**, **HumanMessage**, and **AIMessage**.
- Force **structured JSON** output and parse it safely.
- Build a reusable **LLMChain**.
- Implement **streaming** output and **in-memory + SQLite caching**.
- Build a professional **Streamlit** FinTech dashboard.

5. Technologies

Layer	Technology
Language	Python 3.10+
LLM framework	LangChain (langchain, langchain-openai, langchain-community, langchain-core)
Model provider	OpenAI (e.g. gpt-4o-mini)

User interface	Streamlit
Secrets	python-dotenv (.env file)
Caching	InMemoryCache + SQLiteCache

6. Functional Requirements

- 1 Collect monthly income, ten expense categories, current savings, and a financial goal.
- 2 Compute total expenses, remaining income, savings ratio, and expense ratio in Python.
- 3 Compute a rule-based preliminary score (0-100) in Python.
- 4 Send the numbers to the LLM via an LLMChain and receive structured JSON.
- 5 Parse the JSON safely and render a financial dashboard.
- 6 Stream a written recommendation into the UI.
- 7 Show a financial health score, risk level, priorities, and action plan.
- 8 Offer caching options and a reset-session button.
- 9 Display the educational disclaimer on every relevant screen.

7. System Workflow

- 1 The user enters financial information in the Streamlit form.
- 2 Python calculates total expenses, remaining income, savings ratio, and expense ratio.
- 3 The calculated data is inserted dynamically into the LangChain prompts.
- 4 The LLM analyses the user's financial situation.
- 5 The LLM returns structured JSON.
- 6 The Streamlit app displays the financial dashboard.

8. Input Requirements

- Monthly income (number_input).
- Expenses: housing/rent, food, transportation, utilities, education, healthcare, entertainment, loan/debt, other.
- Current monthly savings (number_input).
- Financial goal (selectbox): save money, emergency fund, pay off debt, vacation, start a business, improve budgeting.
- Currency (selectbox).

9. Python Calculation Requirements

These are **deterministic** — the same inputs always give the same outputs. Implement them in `financial_calculator.py`:

```

total_expenses = sum(all expense categories)
remaining_income = monthly_income - total_expenses
savings_ratio = (savings / monthly_income) * 100
expense_ratio = (total_expenses / monthly_income) * 100
preliminary_score= weighted 0-100 heuristic (savings, leftover,
                                expense ratio, debt burden)

```

Clearly separate these Python calculations from the LLM's insights. Guard against divide-by-zero when income is 0.

10. LangChain Implementation Requirements

Concept	Requirement
ChatOpenAI	Connect the app to the OpenAI model.
SystemMessage	Define the educational financial-assistant role + safety.
HumanMessage	Pass income, expenses, calculations, and goal dynamically.
AIMessage	Show how an assistant reply is represented in a conversation.
PromptTemplate	Reusable single-string template with the financial variables.
ChatPromptTemplate	System + Human messages carrying safety rules and data.
LLMChain	At least one reusable chain that processes the financial data.
JSON output	Return the exact schema; validate and handle invalid JSON.

11. Prompt Engineering Requirements

The PromptTemplate must include these variables: `monthly_income`, `total_expenses`, `remaining_income`, `savings`, `savings_ratio`, `expense_ratio`, `financial_goal`, `expense_breakdown`.

The ChatPromptTemplate must combine system instructions, the financial safety requirements, and the dynamically inserted user data.

12. JSON Output Schema

```

{
  "financial_summary": "",
  "financial_health_score": 0,
  "spending_analysis": [
    { "category": "", "observation": "", "recommendation": "" }
  ],
  "risk_level": "",
  "top_priorities": [],
  "budget_recommendations": [],
  "savings_strategy": [],
  "next_month_action_plan": []
}

```

Financial health score bands (educational only): 80-100 strong, 60-79 generally healthy, 40-59 needs improvement, below 40 high attention.

13. Streaming Requirements

The final recommendations must stream. Use the model's `.stream()` method to yield chunks, and display them live with `st.write_stream()` for a natural typing effect.

```
def stream_recommendations(llm, inputs):
    messages = NARRATIVE_CHAT_TEMPLATE.format_messages(**inputs)
    for chunk in llm.stream(messages):
        if chunk.content:
            yield chunk.content
```

14. Caching Requirements

Implement **both** cache types and add a configuration option to switch between them:

	InMemoryCache	SQLiteCache
Stored in	RAM (memory)	A file on disk (.db)
Speed	Fastest	Fast, slightly slower
Survives restart?	No	Yes
Best for	One session	Reusing across sessions

How it works: `set_llm_cache(...)` registers one global cache; LangChain checks it before each call. Repeating the same prompt is faster and makes no new API call, which reduces cost.

15. Streamlit Dashboard Requirements

Sidebar: logo/title, project info, educational disclaimer, model settings, cache selection, reset-session button.

Main page: organize inputs with columns, expanders, tabs, and a form; submit with a button.

Financial overview: use `st.metric` for income, total expenses, remaining balance, and current savings.

AI analysis: health score (progress bar), spending analysis, risk level, priorities, budget recommendations, savings strategy, and next-month plan — using metrics, progress bars, expanders, info/warning boxes, and tabs.

16. Testing Scenarios

#	Input	Expected calculation	Expected AI response
1	Income 8000, expenses ~2000	Large positive remaining; high savings ratio	High score; LOW risk; growth-focused tips
2	Income 2000, expenses ~2600	Negative remaining; expense ratio >100%	Low score; HIGH risk; urgent cost-cutting
3	Income 5000, debt 2500	High debt share of income	MEDIUM/HIGH risk; debt-reduction priorities
4	Income 4000, savings 1200	Savings ratio ~30%	High score; LOW risk; reinforce good habits
5	Income 3000, expenses 3000	Remaining = 0	MEDIUM/HIGH risk; find room to save

17. Project Folder Structure

```
finwise_ai/
|
|-- app.py                      (Streamlit UI - run this)
|-- requirements.txt
|-- .env.example
|-- README.md
|-- src/
|   |-- __init__.py
|   |-- config.py              (settings + form options)
|   |-- prompts.py             (PromptTemplate + ChatPromptTemplate + JSON)
|   |-- financial_calculator.py(deterministic maths - no AI)
|   |-- chains.py              (ChatOpenAI, LLMChain, streaming)
|   |-- cache_manager.py       (in-memory + SQLite caching)
|   |-- utils.py               (safe JSON parsing + helpers)
|
|-- docs/
|   |-- FinTech_AI_Assignment.pdf
```

18. Step-by-Step Implementation Tasks

- 1 Set up the folder, virtual environment, and install requirements.
- 2 Create `config.py` to load the key and hold form options.
- 3 Implement all calculations in `financial_calculator.py`.
- 4 Write the system prompt and JSON schema in `prompts.py`.
- 5 Build a **PromptTemplate** and a **ChatPromptTemplate** with the financial variables.
- 6 In `chains.py`, build **ChatOpenAI** and a reusable **LLMChain**.
- 7 Add a raw System/Human/AI message demo and a streaming generator.
- 8 Implement in-memory and SQLite caching in `cache_manager.py`.
- 9 Implement **safe JSON parsing** in `utils.py`.
- 10 Build the Streamlit sidebar, form, overview metrics, and AI dashboard.
- 11 Wire streaming into `st.write_stream` and show both scores.
- 12 Add disclaimers everywhere and run the five test scenarios.

19. API Configuration Instructions

- 1 Create an account at platform.openai.com and generate an API key.
- 2 Copy `.env.example` to `.env`.
- 3 Set `OPENAI_API_KEY=sk-...` in the `.env` file.
- 4 Add `.env` to `.gitignore` so the key is never committed.
- 5 The app loads the key with `python-dotenv` at startup.

20. Evaluation Rubric (100 marks)

Criterion	Marks
Streamlit UI and UX	15
Financial calculations	10
ChatOpenAI integration	10
PromptTemplate	10
ChatPromptTemplate and message handling	10
LLMChain	10
Structured JSON output	10
Streaming	10
Caching	10
Code quality and documentation	5
TOTAL	100

21. Bonus Features (optional)

- CSV upload for expense history.
- Expense visualization (bar/pie charts).
- Month-to-month comparison.
- PDF financial-report generation.
- Financial goal tracker.
- Conversation history.
- Dark / light mode.
- Multiple currency support.
- Docker deployment.

22. Submission Requirements

- 1 A GitHub repository containing every file in the structure above.
- 2 A working `requirements.txt` and a `.env.example` (never your real key).
- 3 A README with setup, run instructions, and a caching + Python-vs-AI explanation.
- 4 A short screen recording or live demo showing streaming and the dashboard.
- 5 Well-commented, modular code that runs with `streamlit run app.py`.

Reminder: this project is for education only. It is not financial advice and must not be used to make real investment or money decisions.